

Relatório - Checkpoint 1: Handshake & Sockets

Infraestrutura de Comunicação - 2026.2

Grupo	G4
Matrículas	
Monitor responsável	Danilo Araujo Duleba

1. Contexto e objetivo do Checkpoint 1

O desenvolvimento deste checkpoint partiu de uma implementação cliente-servidor existente, produzida para uma versão de período anterior do trabalho. Essa base já continha o handshake inicial e também mecanismos destinados a etapas posteriores, como fragmentação, controle de integridade, números de sequência, reconhecimentos, retransmissões e simulação de falhas.

Com a especificação do Trabalho I 2026.2, a entrega passou a ser organizada de forma incremental. Por isso, a primeira atividade do grupo foi comparar a base existente com os requisitos atuais, delimitar o que efetivamente pertence ao Checkpoint 1 e preservar separadamente os recursos que serão retomados e revisados nos checkpoints seguintes.

O objetivo desta entrega é estabelecer a conexão TCP entre cliente e servidor e realizar o handshake da aplicação, negociando o modo de operação, o limite máximo do texto inicial e o tamanho da janela. Ao final dessa negociação, a sessão é encerrada, a transmissão da mensagem do usuário ainda não faz parte deste checkpoint.

2. Escopo da entrega e reaproveitamento da implementação anterior

A implementação anterior foi utilizada como referência, mas a versão entregue foi reduzida ao escopo efetivamente avaliado no Checkpoint 1.

Recurso da implementação anterior	Decisão no Checkpoint 1
Sockets e conexão TCP	Mantidos
Negociação do modo de operação	Mantida
Negociação do limite máximo da mensagem	Mantida
Definição da janela pelo servidor	Mantida
Fragmentação da mensagem	Fora do escopo do Checkpoint 1, preservada para evolução posterior
Controle de integridade	Fora do escopo do Checkpoint 1, preservado para evolução posterior
Números de sequência e ACK/NACK	Fora do escopo do Checkpoint 1, preservados para evolução posterior
Retransmissão e timeout	Fora do escopo do Checkpoint 1, preservados para evolução posterior
Simulação de perdas ou corrupção	Fora do escopo do Checkpoint 1, preservada para evolução posterior
Execução efetiva de Go-Back-N/Repetição Seletiva	Fora do escopo do Checkpoint 1, preservada para evolução posterior
Criptografia	Fora do escopo deste checkpoint

threading	Infraestrutura auxiliar mantida no servidor
-----------	---

O modo negociado é apresentado como 1 = envio em lote / Go-Back-N e 2 = envio individual / Repetição Seletiva. Nesta etapa, essa escolha é apenas negociada e armazenada como parâmetro da sessão. O comportamento completo dos algoritmos de transmissão será retomado e validado nos checkpoints posteriores.

3. Especificação do protocolo do Checkpoint 1

3.1 Arquitetura cliente-servidor

A aplicação utiliza sockets IPv4/TCP (AF_INET e SOCK_STREAM). O servidor cria o socket de escuta, executa bind(), listen(5) e accept(), e encaminha cada conexão aceita para uma thread de atendimento. O cliente cria seu socket e inicia a conexão com connect().

O servidor usa SERVER_HOST = "0.0.0.0", escutando nas interfaces IPv4 da máquina. O cliente usa uma constante SERVER_HOST: "127.0.0.1" quando os dois processos estão na mesma máquina ou o endereço IPv4 da máquina servidora quando estão em máquinas diferentes. A configuração é manual, sem descoberta automática.

3.2 Sequência real do handshake

Etapa	Origem	Informação ou ação
1	Cliente → Servidor	Estabelece a conexão TCP.
2	Servidor → Cliente	Solicita o modo de operação e apresenta as opções 1 e 2.
3	Cliente → Servidor	Envia o modo escolhido. Opções inválidas recebem erro e retornam à seleção.
4	Servidor → Cliente	Confirma um modo válido e solicita o limite máximo do texto.
5	Cliente → Servidor	Envia o limite informado. O cliente padrão aceita somente inteiro ≥ 30 .
6	Servidor	Solicita ao operador a janela entre 1 e 5, ENTER seleciona 5.
7	Servidor → Cliente	Envia a informação e o valor numérico da janela.
8	Servidor → Cliente	Valida o tamanho recebido e envia confirmação do handshake ou NEGADO.
9	Cliente e Servidor	Se aceito, o cliente exibe o resumo, em seguida a conexão é encerrada.

A tabela registra a ordem efetivamente implementada nesta versão. O servidor valida novamente o tamanho recebido, independentemente da validação local realizada pelo cliente.

3.3 Formato e delimitação das mensagens

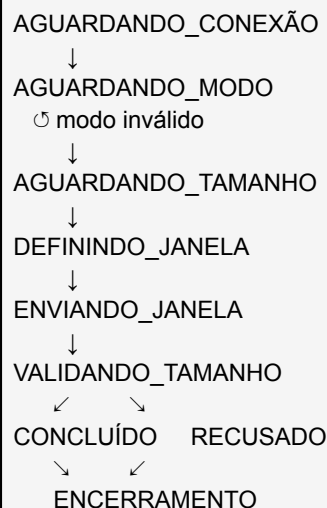
No Checkpoint 1, o protocolo troca mensagens textuais de controle. Cada mensagem é codificada como texto e finalizada pelo byte nulo (\0), utilizado como delimitador de mensagem na camada

de aplicação. A função enviar() utiliza sendall(), e receber() acumula os bytes até localizar o delimitador, preservando eventual conteúdo restante para a próxima leitura.

Tipo de mensagem do CP1	Conteúdo lógico	Enquadramento
Solicitação de modo	Menu com as opções 1 e 2	Texto + \0
Seleção do modo	1 ou 2	Texto + \0
Solicitação do limite	Mensagem de controle	Texto + \0
Limite informado	Inteiro ≥ 30	Texto + \0
Janela	Inteiro entre 1 e 5	Texto + \0
Resultado do handshake	Confirmação ou NEGADO	Texto + \0

Os pacotes de dados com payload, números de sequência, flags, checksum e reconhecimentos ainda não são enviados nesta etapa. O formato completo desses pacotes será retomado na evolução incremental do protocolo.

4. Máquina de estados do handshake (FSM)



A FSM representa somente o Checkpoint 1. Não há estados de transmissão de dados, ACK, NACK, timeout ou retransmissão nesta versão.

5. Análise crítica e adaptações realizadas

5.1 Framing sobre TCP

Problema identificado: a versão anterior utilizava send() e recv(1024) sem delimitação explícita das mensagens. TCP entrega à aplicação um fluxo confiável e ordenado de bytes, mas não preserva as fronteiras definidas pelas mensagens da camada de aplicação.

Impacto: uma leitura poderia conter apenas parte de uma mensagem ou dados de mais de uma mensagem, comprometendo a sequência do handshake.

Decisão do grupo: após revisar o problema e discutir as alternativas, o grupo adotou framing por \0, sendall() e buffer por conexão. O delimitador \n não foi usado porque o menu do modo contém quebras de linha internas que fazem parte da própria mensagem.

Validação: os handshakes executados mantiveram a ordem das mensagens e chegaram ao resumo final ou à recusa esperada.

5.2 Janela inicial

A especificação define janela entre 1 e 5, determinada pelo servidor, com valor inicial 5. O grupo adotou 5 como configuração padrão quando o operador pressiona ENTER, preservando a possibilidade de informar explicitamente outro valor válido entre 1 e 5. Essa foi a interpretação escolhida para representar o valor inicial e, ao mesmo tempo, permitir testes com diferentes janelas.

5.3 Localhost e IP

A revisão mostrou que manter o bind do servidor restrito a localhost não contemplava o cenário de conexão por IPv4 de outra máquina. O grupo ajustou o servidor para escutar em 0.0.0.0 e manteve no cliente a constante SERVER_HOST configurável. A validação desta entrega foi realizada localmente, não se afirma teste entre duas máquinas.

5.4 Revisão assistida por IA e implementação das adaptações

A IA foi utilizada como ferramenta de comparação, revisão e explicação técnica. Ela ajudou a confrontar a implementação anterior com a especificação de 2026.2 e a apontar itens que mereciam análise, como framing sobre TCP, valor inicial da janela, conectividade por IP e coerência do fluxo do handshake.

O grupo avaliou cada ponto em relação ao PDF, ao comportamento real do código e ao escopo do checkpoint. A partir dessa discussão, o grupo decidiu quais mudanças eram pertinentes, realizou as alterações no código e executou os testes necessários para verificar o resultado.

- **Framing:** a análise chamou atenção para a ausência de fronteiras de mensagem, o grupo estudou a causa e implementou a solução com \0.
- **Janela:** a comparação com a especificação levou o grupo a adotar 5 como valor padrão, mantendo a faixa 1–5.
- **Conectividade:** o grupo confirmou que localhost no bind limitava o cenário por IP e implementou a configuração atual.
- **Modo de operação:** a interface foi ajustada para explicitar a nomenclatura adotada pelo projeto sem executar ainda GBN/SR.

5.5 Caso real de debugging durante a adaptação

Durante uma etapa de apoio à redução e comparação de trechos, foi utilizado um recorte de código sugerido com auxílio da IA. Ao aplicar esse recorte, um else: presente no código de referência ficou de fora. Como consequência, o cliente imprimia uma mensagem de recusa logo após um handshake bem-sucedido.

O grupo identificou a contradição durante a execução, comparou o fluxo condicional com a versão de referência, localizou o else: ausente e efetuou a correção. Em seguida, foram testados separadamente o caminho de sucesso e o caminho de recusa. O caso reforçou a necessidade de tratar respostas da IA como insumo de revisão, e não como resultado a ser incorporado automaticamente.

6. Validação e testes

Cenário executado	Resultado
Modo 1, tamanho 30 e janela padrão 5	Aprovado, handshake concluído e parâmetros negociados corretamente.
Modo 2, tamanho 35 e janela 3	Aprovado, handshake concluído com os valores informados.

Entradas abc e 29 para o tamanho	Aprovado, entradas inválidas rejeitadas pelo cliente antes do envio.
Tamanho 10 enviado diretamente ao servidor	Aprovado, servidor respondeu NEGADO e encerrou a conexão.
Resposta de recusa usada para validar o cliente	Aprovado, cliente apresentou a recusa sem informar HANDSHAKE CONCLUÍDO.

Os procedimentos, entradas e saídas detalhadas permanecem registrados em EXEMPLO_TESTE.md. A tabela acima contém apenas os cenários registrados como executados nesta revisão.

7. Manual de utilização

Pré-requisito: Python 3, sem dependências externas.

1. No diretório do projeto, iniciar o servidor com: `python server.py`
2. Em outro terminal, iniciar o cliente com: `python cliente.py`
3. Na mesma máquina, manter `SERVER_HOST = "127.0.0.1"` no cliente. Para máquinas diferentes, substituir por um IPv4 da máquina que executa `server.py`.
4. No cliente, selecionar o modo de operação e informar o limite máximo do texto.
5. No terminal do servidor, definir a janela entre 1 e 5 ou pressionar ENTER para usar o valor padrão 5.
6. O servidor confirma ou recusa o handshake. Se aceito, o cliente exibe modo, limite e janela. Nenhuma mensagem de usuário é transmitida neste checkpoint.

8. Processo de construção e uso de IA

A estratégia adotada segue o princípio da especificação de utilizar a IA como catalisadora do aprendizado. A ferramenta foi usada para ampliar a análise, questionar a aderência da implementação à nova especificação, explicar conceitos e apoiar a revisão, a responsabilidade técnica e conceitual permaneceu com o grupo.

8.1 Estratégia de uso da IA

O fluxo de trabalho adotado foi: implementação existente → leitura da especificação → comparação assistida por IA → discussão dos apontamentos pelo grupo → decisão técnica → implementação pelo grupo → testes → nova revisão.

No contexto de transporte confiável, a IA foi usada para ajudar a distinguir os serviços já fornecidos pelo TCP das regras de confiabilidade que o trabalho exigirá posteriormente na camada de aplicação. No Checkpoint 1, isso foi especialmente importante para compreender que o TCP fornece um fluxo de bytes, mas o protocolo da aplicação ainda precisa definir suas próprias mensagens e fronteiras.

8.2 Aprendizado técnico

- **Estabelecimento da conexão:** socket IPv4/TCP, bind, listen, accept e connect.
- **Troca de dados:** natureza de fluxo do TCP, `sendall()`, `recv()` e framing por `\0`.
- **Protocolo de aplicação:** modo, limite máximo e janela como regras negociadas sobre a conexão TCP.
- **Modelagem:** formalização da sequência do handshake, da FSM do Checkpoint 1 e do formato das mensagens de controle.

8.3 Análise, decisão e implementação pelo grupo

A análise assistida ajudou a separar o que já atendia ao checkpoint do que precisava ser revisto. O grupo discutiu cada apontamento antes de modificar a base. Entre as decisões tomadas estiveram preservar a estrutura de sockets e o handshake, manter threading como infraestrutura auxiliar, adotar framing explícito, ajustar a configuração de rede e representar o valor inicial da janela.

As mudanças escolhidas foram realizadas pelo grupo. Depois disso, cliente e servidor foram executados, os parâmetros negociados foram verificados e os cenários da seção 6 foram usados para confirmar os caminhos de aceitação e recusa.

8.4 Análise crítica e debugging

O caso do else: ausente foi o principal exemplo de validação crítica. Uma sugestão de recorte/cópia de trechos, usada durante a redução da base, deixou de preservar uma parte da estrutura condicional. O grupo percebeu o comportamento contraditório ao executar o programa, investigou a diferença em relação ao código de referência, corrigiu o trecho e retestou os dois caminhos.

Esse episódio mostrou que a IA não foi tratada como fonte infalível: suas sugestões foram avaliadas, e o comportamento observado em teste e a especificação oficial foram usados para decidir se uma alteração deveria ou não permanecer.

8.5 Diário de aprendizado

Etapas	Uso da IA	Atuação do grupo	Aprendizado/resultados
1. Delimitação do escopo	Comparação entre PDF e base antiga	Leitura da especificação e decisão do que manter	Separação entre handshake do CP1 e recursos posteriores
2. Revisão dos sockets	Explicação e apontamento de riscos	Estudo do fluxo TCP e escolha da solução	Compreensão de framing e de sendall/recv
3. Modelagem do protocolo	Apoio para organizar sequência/FSM	Validação da ordem real no código	FSM e formato das mensagens do handshake
4. Adaptação	Revisão das decisões propostas	Implementação das alterações selecionadas	Janela padrão, IP configurável e nomenclatura do modo
5. Testes e debugging	Apoio na auditoria pós-alteração	Execução dos cenários, identificação e correção do else	Validação crítica das sugestões da IA

8.6 Registro de prompts

Como registro do processo, os prompts abaixo consolidam os principais tipos de interação utilizados. Eles foram organizados a partir das conversas de revisão e comparação realizadas durante o checkpoint.

Prompt 1 - Comparação e delimitação do escopo

Análise a especificação do Trabalho I 2026.2 junto com esta implementação anterior. O projeto antigo possui funcionalidades além do primeiro checkpoint. Identifique quais partes correspondem especificamente ao Checkpoint 1, quais pertencem às etapas posteriores e quais diferenças existem entre a implementação atual e a nova especificação. Não altere o código, quero primeiro compreender o que precisamos manter e adaptar.

Prompt 2 - Revisão técnica da base

Revise o trecho de sockets e handshake que será reaproveitado. Explique seu funcionamento e procure fragilidades que possam afetar os requisitos atuais, especialmente conexão cliente-servidor, troca de mensagens, modo, limite do texto e janela. Diferencie problemas reais, melhorias opcionais e pontos que já estão adequados. Não modifique nada nesta etapa.

Prompt 3 - Avaliação das decisões do grupo

Com base nos pontos encontrados, avalie nossas decisões para o Checkpoint 1: manter o handshake existente, usar framing explícito sobre TCP, adotar 5 como valor padrão da janela, manter a faixa 1–5 e permitir localhost/IP. Questione qualquer decisão que esteja tecnicamente errada ou não corresponda ao PDF. Queremos validar as escolhas antes de implementá-las.

Prompt 4 - Auditoria após a implementação

As adaptações discutidas já foram efetuadas no projeto. Compare o código resultante com a especificação e verifique se o Checkpoint 1 está atendido. Execute os cenários possíveis, procure inconsistências entre código, README e relatório e aponte problemas antes da entrega. Não faça novas alterações automaticamente.

9. Limitações atuais e evolução incremental

O Checkpoint 1 limita-se à conexão e à negociação inicial. Go-Back-N e Repetição Seletiva são selecionados no handshake, mas ainda não são executados. Também não fazem parte desta versão a fragmentação do texto, os números de sequência, ACK/NACK, checksum, retransmissões, timeout ou simulação de perdas e corrupção.

O servidor mantém a estrutura com threads da implementação existente. Os testes desta entrega foram realizados com um cliente por vez, concorrência entre múltiplas sessões interativas não foi objeto da validação atual.

Nos checkpoints seguintes, os recursos preservados da implementação anterior serão retomados e revisados sobre a base de conexão e handshake validada nesta entrega.